



Hands On Multithreading and Multiprocessing Lab 1 Report

Bora Yalçın

1 Overview

This report was written for the first hands on multithreading and multiprocessing lab. Programming languages I preferred using while going through exercises are Java and mostly C++. The reason behind why I used Java at some exercises is because I find Java easier to read and understand when it comes to Object-Oriented Programming and also makes it easier to implement due to how it's built. By the end of this report we should be able to implement and understand programs using multithreading and parallel processes, distinguish concurrency from parallelism, handle common concurrency problems such as race conditions and deadlocks, use synchronization mechanisms such as mutexes, semaphores, condition variables, barriers and explain whether and at what point parallelism and concurrency improves overall performance and be used.

2 Tools and Hardware Used

2.1 Hardware

- **CPU:** Intel Core Ultra 9 285H (16 logical processors)
- **GPU:** NVIDIA GeForce RTX 5070 Laptop GPU (8GB VRAM)
- **RAM:** 32GB DDR5

2.2 Operating System

- Kubuntu 26.04.1 LTS (Resolute Raccoon)
- Kernel: Linux 7.0.0-31-generic
- Desktop Environment: KDE Plasma 6.6.6 (Wayland)
- NVIDIA Driver Version: 595.84

2.3 Programming Languages and Runtimes

- C++ (GCC 15.2.0 / G++ 15.2.0)
- Java (OpenJDK 25.0.2, managed via SDKMAN)
- Python 3.14.4

2.4 Editors and IDEs

- Visual Studio Code
- Neovim (LazyVim configuration)

2.5 Compilers and Build Tools

- GCC / G++ 15.2.0 (Ubuntu build)
- Clang
- CMake
- Make / Ninja

3 Exercise 1 - Simple Thread Creation

In this exercise I have written a simple thread creating program. I've actually implemented the first few exercises both in Java and C++, but to see both implementations you can checkout the related GitHub repository I have given below at the end of the report.

Listing 1: ex1.cpp

```
1  #include <iostream>
2  #include <thread>
3
4  void printRange(int start, int end, int threadNum) {
5      for(int i = start; i<=end; ++i){
6          std::cout << "Thread " << threadNum << ':' << i << std::endl;
7      }
8  }
9
10 int main(){
11
12     std::thread t1(printRange,1,5,1);
13     std::thread t2(printRange,6,10,2);
14
15     t1.join();
16     t2.join();
17
18     std::cout <<"Both threads are finished !" << std::endl;
19
20     return 0;
21     // to compile this file run g++ -std=c++20 -O2 -pthread ex1.cpp -o ex1
22 }
23
```

What we've done in this program is pretty simple. Using the C++ thread library, we have created 2 threads that will work on the function called `printRange`; in order to do that we just give the function parameters and its name. I ran the program 10 times in a row and in none of those runs did I get the exact same ordering twice, so below I am showing two representative runs out of them.

Listing 2: First run

```
1  Thread 2:6
2  Thread 2:7
3  Thread 2:8
```

```

4      Thread 2:9
5      Thread 2:10
6      Thread 1:1
7      Thread 1:2
8      Thread 1:3
9      Thread 1:4
10     Thread 1:5
11     Both threads are finished!
12

```

Listing 3: Second run

```

1      Thread 2:6
2      Thread 2:7
3      Thread 1:1
4      Thread 1:2
5      Thread 1:3
6      Thread 1:4
7      Thread 1:5
8      Thread 2:8
9      Thread 2:9
10     Thread 2:10
11     Both threads are finished!
12

```

Now let's answer some questions:

Question a As you guys have noticed, we do not always get the same output order. This is caused by how threads work: since threads run concurrently and not sequentially, they do not wait for each other, and when a task is completed it is instantly printed before the other thread finishes its own tasks.

Question b The answer to this question lies behind how the OS scheduler and threads work, since a thread can be interrupted mid-execution through a context switch and the scheduler can give CPU time to the other thread at any point, they can end up interleaved.

Question c No, interleaving does not necessarily prove the threads executed at the same instant, since a single core can also produce interleaved output through fast time-slicing, where threads simply take turns rather than truly running simultaneously.

Question d This behavior comes directly from how the OS scheduler and time-sharing work, since each thread only gets a small time quantum before a context switch happens, and this constant switching is what creates interleaving without guaranteeing real parallel execution.

4 Exercise 2 - Does Multithreading Always Improve Performance?

4.1 Part A - One Blocking Task

For this part, I implemented the example given in the lab document, comparing a sequential call to a blocking task against the same task run in a single additional thread. I wrote it both in Java and in C++, and the C++ version using `std::this_thread::sleep_for` and `std::chrono` is the one shown here.

Listing 4: MultithreadingExample.cpp

```

1      #include <iostream>
2      #include <thread>

```

```

3     #include <chrono>
4
5     void performTask(long milliseconds){
6         std::this_thread::sleep_for(std::chrono::milliseconds(milliseconds));
7     }
8
9     int main(){
10        // Sequential execution
11        auto start = std::chrono::steady_clock::now();
12        performTask(2000);
13        auto end = std::chrono::steady_clock::now();
14        std::chrono::duration<double> seqElapsed = end - start;
15        std::cout << "Sequential time: " << seqElapsed.count() << " s" << std::endl;
16
17        // Execution using one additional thread
18        start = std::chrono::steady_clock::now();
19        std::thread t(performTask, 2000);
20        t.join();
21        end = std::chrono::steady_clock::now();
22        std::chrono::duration<double> threadElapsed = end - start;
23        std::cout << "One-thread time: " << threadElapsed.count() << " s" << std::
endl;
24
25        return 0;
26    }
27

```

Listing 5: Output

```

1     Sequential time: 2.000 s
2     One-thread time: 2.002 s
3

```

As we can see from the output the two results are almost identical, so adding a thread here does not speed anything up. The reason behind this is that `main` is actually a thread itself, so when we call `join()` it forces `main` to wait anyway, and since there is only ONE unit of work there is nothing to actually run in parallel. The tiny extra 0.002s is just the cost of creating one OS thread.

Question a As we have just discussed above the two results are almost identical since `main` itself is a thread and no actual parallel work is going on, because `join()` tells `main` to wait anyway.

Question b No, adding a single thread in this case doesn't make the program any faster, in fact it is actually slower by about 0.002 seconds due to the cost of creating one OS thread. And adding even more threads would not help either here, because there is only one 2 second task to do and you cannot split a single sleep between workers.

Question c Calling `join()` right after `start()` is exactly the reason why there is no improvement in time, since it tells `main` to sit and wait while our thread finishes its work. That turns the whole thing back into something sequential.

Question d In this case the overhead was the cost of creating a single OS thread, which cost us about 0.002 seconds.

4.2 Part B - Several Independent Blocking Tasks

Here, 50 independent 200ms tasks were distributed across $p \in \{1, 2, 3, 4, 5\}$ worker threads, with each configuration repeated and averaged.

Listing 6: MultithreadingExampleWorkers.cpp

```
1  #include <iostream>
2  #include <thread>
3  #include <chrono>
4  #include <vector>
5
6  const int TOTAL_TASKS = 50;
7  const long WAIT_MS = 200;
8
9  void performTask() {
10     std::this_thread::sleep_for(std::chrono::milliseconds(WAIT_MS));
11 }
12
13 void runWithWorkers(int p) {
14     std::vector<std::thread> workers;
15
16     int tasksPerWorker = TOTAL_TASKS / p;
17     int remainder = TOTAL_TASKS % p;
18
19     for (int i = 0; i < p; ++i) {
20         int tasksForThisWorker = tasksPerWorker + (i < remainder ? 1 : 0);
21
22         workers.emplace_back([tasksForThisWorker]() {
23             for (int j = 0; j < tasksForThisWorker; ++j) {
24                 performTask();
25             }
26         });
27     }
28
29     for (auto& t : workers) t.join();
30 }
31
32 int main() {
33     std::vector<int> workerCounts = {1, 2, 3, 4, 5};
34     int repetitions = 2;
35
36     for (int p : workerCounts) {
37         double totalTime = 0.0;
38
39         for (int r = 0; r < repetitions; ++r) {
40             auto start = std::chrono::steady_clock::now();
41             runWithWorkers(p);
42             auto end = std::chrono::steady_clock::now();
43
44             std::chrono::duration<double> elapsed = end - start;
45             totalTime += elapsed.count();
46         }
47
48         double avgSeconds = totalTime / repetitions;
49         std::cout << "p=" << p << " -> avg time: " << avgSeconds << " s" << std:::endl;
50     }
51 }
```

```

52     return 0;
53 }
54

```

p	T_p (s)	$S_p = T_1/T_p$	$E_p = S_p/p$
1	10.0071	1.00	1.00
2	5.0032	2.00	1.00
3	3.40258	2.94	0.98
4	2.60268	3.85	0.96
5	2.00262	5.00	1.00

Table 1: Execution time, speedup, and efficiency for Exercise 2 Part B

The table shows that S_p and E_p values are close to ideal ($S_p \approx p$, $E_p \approx 1.0$). It is mainly because the 50 tasks are completely independent and I/O-bound (just sleeping), so there is no actual CPU contention among threads. The speedup does not come from extra CPU power, it comes from overlapping sleep times. If `performTask()` was a CPU intensive function instead of just sleeping, the threads would compete for the CPU and E_p would drop.

Question a S_p and E_p are calculated as $S_p = T_1/T_p$ and $E_p = S_p/p$; for the actual values please refer to Table 1.

Question b The execution time goes down because the 50 tasks are independent of each other, so instead of one worker doing 50 sleeps one after another, p workers each sit through only $50/p$ of them and all of them are waiting at the same time.

Question c No, the speedup does not come from extra CPU power. Our `performTask()` is I/O-bound, it just sleeps and does not use the CPU at all while waiting, so what we are actually overlapping here is waiting time and not computation.

Question d If it was a CPU intensive work the threads would actually need the CPU and they would compete over it, which would make E_p drop well below 1.0. We see exactly this later in Exercise 7 with matrix multiplication.

4.3 Part C - Performance Curves

Below here we can see the related performance curves which are generated by a Python script that you can find in the GitHub repository.

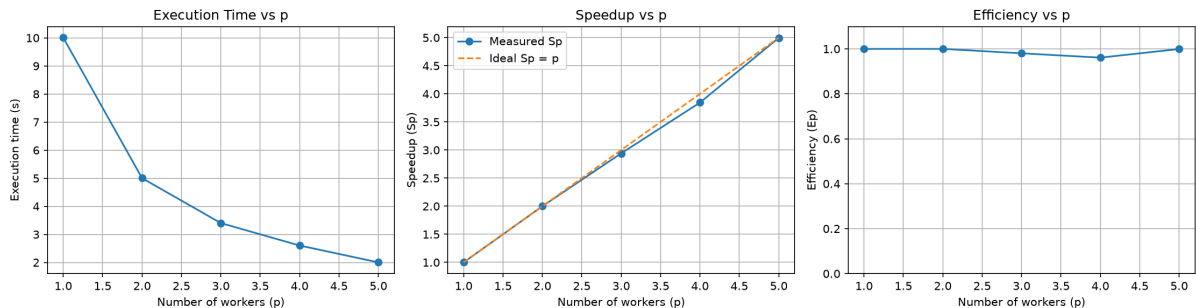


Figure 1: Execution time, speedup (vs. ideal $S_p = p$), and efficiency as a function of p

So what are these curves telling us? Since we have used the measurements from Part B, we can see that T_p drops hyperbolically ($1/p$ shaped), S_p tracks the ideal $S_p = p$ line almost exactly, and E_p stays close to 1.0 across the whole graph. There is a small dip at $p = 4$ which is most

likely just OS scheduling jitter and background processes on my laptop rather than anything structural.

5 Exercise 3 - Deadlock Simulation and Prevention

5.1 Part 1 - Creating the Deadlock

Two lock objects, L1 and L2, are used by two threads that acquire them in *opposite* order: Thread 1 locks L1 then waits for L2, while Thread 2 locks L2 then waits for L1. I implemented this one in Java using `synchronized` blocks, since the monitor-based syntax makes the nesting of the two locks very easy to read. You can also find the C++ version of this exercise (using `std::mutex` and `lock_guard`) in the repository.

Listing 7: Deadlock.java

```
1  public class Deadlock {
2
3      static final Object L1 = new Object();
4      static final Object L2 = new Object();
5
6      static void thread1() {
7          synchronized (L1) {
8              System.out.println("Thread 1 acquired L1");
9              try { Thread.sleep(100); } catch (InterruptedException e) {}
10
11              System.out.println("Thread 1 waiting for L2...");
12              synchronized (L2) {
13                  System.out.println("Thread 1 acquired L2");
14              }
15          }
16      }
17
18      static void thread2() {
19          synchronized (L2) {
20              System.out.println("Thread 2 acquired L2");
21              try { Thread.sleep(100); } catch (InterruptedException e) {}
22
23              System.out.println("Thread 2 waiting for L1...");
24              synchronized (L1) {
25                  System.out.println("Thread 2 acquired L1");
26              }
27          }
28      }
29
30      public static void main(String[] args) throws InterruptedException {
31          Thread t1 = new Thread(Deadlock::thread1);
32          Thread t2 = new Thread(Deadlock::thread2);
33
34          t1.start();
35          t2.start();
36
37          t1.join();
38          t2.join();
39
40          System.out.println("Both threads finished (should never print!)");
41      }
42  }
43
```

Running this with `timeout 5 java Deadlock; echo $?` confirmed the program never terminates on its own: it was killed by the timeout, returning exit code 124, and the final "Both threads finished" line was never printed. Both threads print that they acquired their first lock, then both print that they are waiting for the second one, and then nothing else happens.

5.2 Part 2 - Preventing the Deadlock

The fix applied here is the *consistent global lock ordering* strategy: both threads now acquire L1 first and L2 second.

Listing 8: DeadlockFixed.java

```
1  public class DeadlockFixed {
2
3      static final Object L1 = new Object();
4      static final Object L2 = new Object();
5
6      static void thread1() {
7          synchronized (L1) {
8              System.out.println("Thread 1 acquired L1");
9              try { Thread.sleep(100); } catch (InterruptedException e) {}
10
11              System.out.println("Thread 1 waiting for L2...");
12              synchronized (L2) {
13                  System.out.println("Thread 1 acquired L2");
14              }
15          }
16      }
17
18      static void thread2() {
19          // FIX: same order as thread1 - L1 first, then L2
20          synchronized (L1) {
21              System.out.println("Thread 2 acquired L1");
22              try { Thread.sleep(100); } catch (InterruptedException e) {}
23
24              System.out.println("Thread 2 waiting for L2...");
25              synchronized (L2) {
26                  System.out.println("Thread 2 acquired L2");
27              }
28          }
29      }
30
31      public static void main(String[] args) throws InterruptedException {
32          Thread t1 = new Thread(DeadlockFixed::thread1);
33          Thread t2 = new Thread(DeadlockFixed::thread2);
34
35          t1.start();
36          t2.start();
37
38          t1.join();
39          t2.join();
40
41          System.out.println("Both threads finished successfully.");
42      }
43  }
```

After this fix, the program always terminates and prints "Both threads finished successfully."

5.3 Part 3 - Analysis

Lets answer some questions regarding this exercise!

Question a So why did the deadlock occur? Each thread held one lock while waiting for the other thread's lock, and neither one would release the lock it was currently holding. Thread 1 was sitting on L1 waiting for L2, and Thread 2 was sitting on L2 waiting for L1, so both of them were waiting for something that would never be freed.

Question b The conditions that made this deadlock possible are the four classic ones, and all of them were present in my code at the same time. First, *mutual exclusion*, since only one thread can hold a monitor at a time. Second, *hold and wait*, because each thread kept its first lock while asking for the second one. Third, *no preemption*, since nobody can forcefully take a lock away from a thread that holds it. And finally *circular wait*, which is the one I created on purpose by making the two threads take the locks in opposite order.

Question c To prevent the deadlock I used the consistent global lock ordering strategy, which means I forced both threads to always take L1 first and L2 second. This kills the circular wait condition, because if everybody asks for the locks in the same order there is no way to build a cycle anymore.

Question d We may have prevented the deadlock but there is a trade-off here. Once both threads want L1 first, whichever one gets it blocks the other one completely until it is fully done, so while one is working the other is just waiting and doing nothing. In this particular example that means we lost almost all of the concurrency we wanted in the first place, and the execution time is basically the same as running the two functions sequentially. In a real program you would keep the critical sections as small as possible to reduce this cost.

6 Exercise 4 - Race Condition and Thread Synchronization Using a Mutex

A shared counter is incremented by one thread and decremented by another, first without any synchronization and then protected by a lock, for several iteration counts. The Java version is shown here; the C++ version (using `std::mutex` and `lock_guard`) is in the repository.

Listing 9: RaceCondition.java

```
1  class RaceCondition {
2
3      static long counter = 0;
4      static final Object lock = new Object();
5
6      static void incrementWithoutSync(long number){
7          for(int i = 0; i < number; i++){
8              counter++;
9          }
10     }
11
12     static void decrementWithoutSync(long number){
13         for(int i = 0; i < number; i++){
14             counter--;
15         }
16     }
17
18     static void incrementWithSync(long number){
19         for (int i = 0; i < number; i++){
```

```

20         synchronized (lock) {
21             counter++;
22         }
23     }
24 }
25
26 static void decrementWithSync(long number){
27     for (int i = 0; i < number; i++){
28         synchronized (lock) {
29             counter--;
30         }
31     }
32 }
33
34 static double run(long number, boolean isSync) throws InterruptedException{
35
36     // reset counter to 0 at the start of each test
37     counter = 0;
38     long start = System.nanoTime();
39
40     Thread t1;
41     Thread t2;
42
43     if(isSync){
44         t1 = new Thread(() -> incrementWithSync(number));
45         t2 = new Thread(() -> decrementWithSync(number));
46     }else{
47         t1 = new Thread(() -> incrementWithoutSync(number));
48         t2 = new Thread(() -> decrementWithoutSync(number));
49     }
50     t1.start();
51     t2.start();
52
53     t1.join();
54     t2.join();
55
56     long end = System.nanoTime();
57     double timeElapsedInSeconds = (end - start) / 1_000_000_000.0;
58
59     if(isSync){
60         System.out.println("Number of Sync Iterations = " + number + ", Final
Counter = " + counter + ", Time Elapsed (seconds) = " + timeElapsedInSeconds);
61     }else{
62         System.out.println("Number of NonSync Iterations = " + number + ", Final
Counter = " + counter + ", Time Elapsed (seconds) = " + timeElapsedInSeconds);
63     }
64
65     return timeElapsedInSeconds;
66 }
67
68 public static void main(String[] args) throws InterruptedException{
69
70     run(10_000, false);
71     run(10_000, true);
72
73     System.out.println("-----");
74
75     run(100_000, false);

```

```

76         run(100_000, true);
77
78         System.out.println("-----");
79
80         run(1_000_000, false);
81         run(1_000_000, true);
82     }
83 }
84

```

Listing 10: Output (Java)

```

1  Number of NonSync Iterations = 10000, Final Counter = -1717, Time Elapsed (
   seconds) = 0.005841923
2  Number of Sync Iterations = 10000, Final Counter = 0, Time Elapsed (seconds) =
   0.003404069
3  -----
4  Number of NonSync Iterations = 100000, Final Counter = 13332, Time Elapsed (
   seconds) = 0.001474596
5  Number of Sync Iterations = 100000, Final Counter = 0, Time Elapsed (seconds) =
   0.017171113
6  -----
7  Number of NonSync Iterations = 1000000, Final Counter = -1054, Time Elapsed (
   seconds) = 0.001893177
8  Number of Sync Iterations = 1000000, Final Counter = 0, Time Elapsed (seconds)
   = 0.13367981
9

```

n (iterations)	Unsync. counter	Sync. counter	Sync. time / Unsync. time
10,000	-1717	0	$\approx 0.6\times$
100,000	13332	0	$\approx 12\times$ slower
1,000,000	-1054	0	$\approx 71\times$ slower

Table 2: Race condition results (Java): unsynchronized final counter values compared to the always-correct synchronized version

One important detail worth mentioning came from the C++ version of the same program. When it was first compiled with `-O2`, the unsynchronized version consistently returned 0, because the compiler collapsed the whole increment/decrement loop into a single register operation per thread (undefined behavior on an unsynchronized shared variable gives the compiler this freedom). Recompiling with `-O0` restored the expected non-deterministic, incorrect results. Java did not show this effect since the JVM does not do that particular whole-loop collapse here. This was actually a nice surprise for me, because it means an optimized build can accidentally hide a race condition that is definitely still there in the code.

Question a Theoretically we should always end up with 0, since both threads do the exact same number of operations on the same counter: one of them increments it n times and the other one decrements it n times, so everything they do should cancel each other out perfectly.

Question b No, and this is the whole point of the exercise. As you can see in Table 2, without synchronization we got -1717, 13332 and -1054 instead of 0. What makes it even more interesting is that running the exact same program again does not give the same wrong value either: an earlier run of the 10,000-iteration case happened to give 0 purely by chance, while the run shown in the table gave -1717. So the result is not just wrong, it is unpredictably

wrong, which is much more dangerous because a quick test might accidentally pass and hide the bug.

Question c The reason behind this lies in the fact that `counter++` is not a single operation even though it looks like one. At the machine level it is actually three separate steps: read the current value of the counter into a register, add 1 to it, and write the new value back to memory. Nothing stops the scheduler from interrupting a thread in the middle of these three steps. So if the incrementing thread reads the value 5, and before it writes 6 back the decrementing thread also reads 5 and writes 4, then the increment is completely lost since the second thread overwrote it with a result calculated from a stale value. This is exactly why we lose operations and end up with a random final number.

Question d The `synchronized` block solves this by making that read-modify-write sequence atomic. Once a thread enters the critical section it holds the lock, and the other thread simply cannot enter until the lock is released. This means no thread can ever observe or overwrite a half-finished update, so every single increment and decrement is actually applied and we always get exactly 0.

Question e Correctness is not free though. Looking at the table again, at 1,000,000 iterations the synchronized version took about 71 times longer than the unsynchronized one, since every single iteration now has to acquire and release the lock instead of just touching a variable. At 10,000 iterations the synchronized version actually came out slightly faster, but that is just the JVM's JIT compiler not having warmed up yet on the very first run rather than a real performance win. The general trend is clear: the more iterations we do, the more the locking overhead adds up.

7 Exercise 5 - Producer-Consumer Problem with Threads

Instead of a plain bounded queue, I implemented a small *inventory* model: a fixed number of slots (`NUM_SLOTS`), where each slot is assigned to one item type and has its own quantity/capacity limit. Multiple producers add different item types, and multiple consumers remove items. The Java version is shown here, using `ReentrantLock` together with two separate `Condition` objects; the C++ version in the repository does the same thing with `std::mutex` and two `std::condition_variable` objects.

Listing 11: `ProducerConsumer.java`

```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.concurrent.locks.Condition;
4  import java.util.concurrent.locks.ReentrantLock;
5
6  class ProducerConsumer {
7
8      final static int NUM_SLOTS = 5;
9
10     static ReentrantLock lock = new ReentrantLock();
11     static Condition slotAvailable = lock.newCondition();
12     static Condition itemAvailable = lock.newCondition();
13
14     static int activeProducers = 0;
15
16     static class Slot {
17         String itemType = "";
18         int quantity = 0;
19         static int slotLimit = 20;
```

```

20     }
21
22     static ArrayList<Slot> inventory = new ArrayList<>();
23
24     static int findSlotFor(String itemType) {
25         int emptySlot = -1;
26         for (int i = 0; i < inventory.size(); i++) {
27             Slot s = inventory.get(i);
28             if (s.itemType.equals(itemType) && s.quantity < s.slotLimit) {
29                 return i;
30             }
31             if (s.itemType.equals("") && emptySlot == -1) {
32                 emptySlot = i;
33             }
34         }
35         return emptySlot;
36     }
37
38     static void producer(int producerId, String itemType, int itemCount) throws
InterruptedException {
39         for (int i = 0; i < itemCount; i++) {
40             lock.lock();
41             try {
42                 while (findSlotFor(itemType) == -1) {
43                     slotAvailable.await();
44                 }
45                 int index = findSlotFor(itemType);
46                 inventory.get(index).itemType = itemType;
47                 inventory.get(index).quantity++;
48
49                 System.out.println("[Producer " + producerId + "] added " + itemType
50                     + " -> slot " + index + " (id=" + index + "-" + inventory.get(index).
quantity
51                     + "), quantity now=" + inventory.get(index).quantity + "/" + inventory.
get(index).slotLimit);
52
53                 itemAvailable.signal();
54             } finally {
55                 lock.unlock();
56             }
57             Thread.sleep(50);
58         }
59
60         lock.lock();
61         try {
62             activeProducers--;
63             if (activeProducers == 0) {
64                 itemAvailable.signalAll();
65             }
66         } finally {
67             lock.unlock();
68         }
69     }
70
71     static void consumer(int consumerId) throws InterruptedException {
72         while (true) {
73             int idx = -1;
74             String consumedType = null;

```

```

75         int remaining = 0;
76
77         lock.lock();
78         try {
79             while (allEmpty() && activeProducers > 0) {
80                 itemAvailable.await();
81             }
82
83             // which slot has items
84             for (int i = 0; i < inventory.size(); i++) {
85                 if (inventory.get(i).quantity > 0) {
86                     idx = i;
87                     break;
88                 }
89             }
90
91             if (idx == -1) {
92                 // no item left and all producers are done
93                 break;
94             }
95
96             inventory.get(idx).quantity--;
97             consumedType = inventory.get(idx).itemType;
98             remaining = inventory.get(idx).quantity;
99
100             if (remaining == 0) {
101                 inventory.get(idx).itemType = "";
102             }
103
104             System.out.println("    [Consumer " + consumerId + "] took " +
consumedType
105                 + " from slot " + idx + ", quantity now=" + remaining);
106
107             slotAvailable.signal();
108         } finally {
109             lock.unlock();
110         }
111
112         Thread.sleep(70);
113     }
114 }
115
116 static boolean allEmpty() {
117     for (Slot s : inventory) {
118         if (s.quantity > 0) return false;
119     }
120     return true;
121 }
122
123 public static void main(String[] args) throws InterruptedException {
124
125     activeProducers = 3;
126
127     for (int i = 0; i < NUM_SLOTS; i++) {
128         inventory.add(new Slot());
129     }
130
131     List<Thread> producers = new ArrayList<>();

```

```

132     producers.add(new Thread(() -> {
133         try { producer(0, "apple", 8); } catch (InterruptedException e) {}
134     }));
135     producers.add(new Thread(() -> {
136         try { producer(1, "pear", 8); } catch (InterruptedException e) {}
137     }));
138     producers.add(new Thread(() -> {
139         try { producer(2, "banana", 8); } catch (InterruptedException e) {}
140     }));
141
142     List<Thread> consumers = new ArrayList<>();
143     for (int i = 0; i < 2; i++) {
144         int consumerId = i;
145         consumers.add(new Thread(() -> {
146             try { consumer(consumerId); } catch (InterruptedException e) {}
147         }));
148     }
149
150     for (Thread t : producers) t.start();
151     for (Thread t : consumers) t.start();
152
153     for (Thread t : producers) t.join();
154     for (Thread t : consumers) t.join();
155
156     System.out.println("All producers and consumers finished.");
157 }
158 }
159

```

The system guarantees that a producer never inserts into a full slot and a consumer never removes from an empty inventory, and correctly handles three producers (each supplying a different item type) and two consumers running concurrently. The C++ version, using `std::condition_variable` instead of `Condition` objects, produces the same behaviour. A sample of the output is given below.

Listing 12: Output (excerpt)

```

1    [Producer 0] added apple -> slot 0 (id=0-1), quantity now=1/20
2    [Producer 1] added pear -> slot 1 (id=1-1), quantity now=1/20
3    [Consumer 0] took apple from slot 0, quantity now=0
4    [Consumer 1] took pear from slot 1, quantity now=0
5    [Producer 2] added banana -> slot 0 (id=0-1), quantity now=1/20
6    [Producer 1] added pear -> slot 1 (id=1-1), quantity now=1/20
7    [Producer 0] added apple -> slot 2 (id=2-1), quantity now=1/20
8    [Producer 2] added banana -> slot 0 (id=0-2), quantity now=2/20
9    [Consumer 0] took banana from slot 0, quantity now=1
10   [Consumer 1] took banana from slot 0, quantity now=0
11   ...
12   [Consumer 0] took apple from slot 2, quantity now=0
13   All producers and consumers finished.
14

```

The two conditions the implementation has to respect follow directly from this structure. A producer looks for a slot with `findSlotFor()`, which first tries to find an existing, non-full slot of the same item type, and only then falls back to a completely free slot; if neither exists it returns `-1` and the producer waits on `slotAvailable`. A consumer scans for any slot with a positive quantity, and if the whole inventory is empty it waits on `itemAvailable`. When a slot's quantity drops back to 0 its type is cleared, which releases that slot so a different item type can

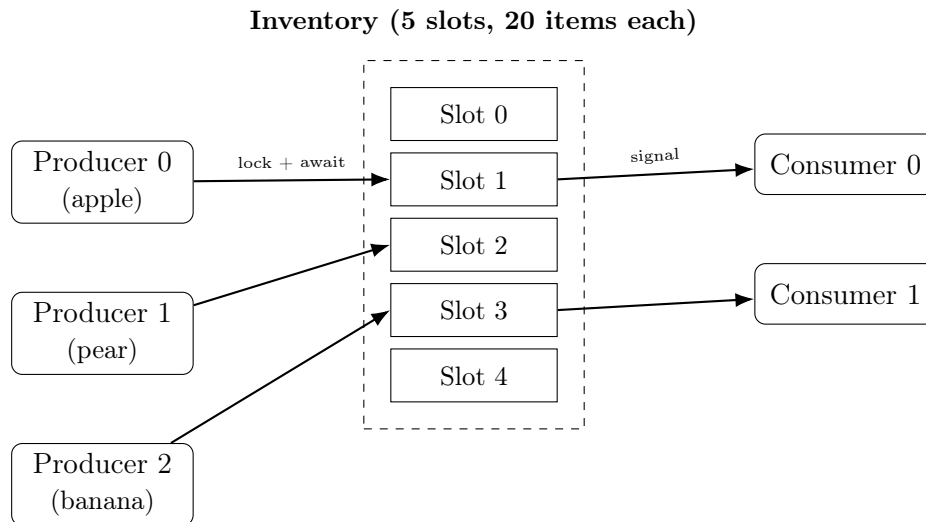


Figure 2: Structure of the inventory-based producer–consumer system. Each producer supplies one item type; the inventory holds at most 5 distinct item types, each limited to 20 items. Producers block on `slotAvailable` when no suitable slot exists, consumers block on `itemAvailable` when the whole inventory is empty.

Slot	itemType	quantity	slotLimit	State
0	"banana"	2	20	In use, accepts more bananas
1	"pear"	20	20	In use, full – producer must wait
2	"apple"	7	20	In use, accepts more apples
3	" "	0	20	Free – can be claimed by any new item type
4	" "	0	20	Free – can be claimed by any new item type

Table 3: Example snapshot of the inventory state. A producer of a new item type can only start if at least one slot is free; a producer of an existing type can only add if that type’s slot is below its limit.

use it later.

One limitation I noticed while testing is that the id printed in the log (`slot-quantity`) is not a truly unique item id. Once a slot empties completely and gets refilled, the quantity starts from 1 again, so the same id can appear twice for two different items. Fixing it properly would need a separate global counter that never resets, which I left out to keep the example simple.

8 Exercise 6 - Basic Multiprocessing Task

Several child processes are created with `fork()`, each computing the factorial of one number from a list and returning the result to the parent through its own dedicated pipe.

Listing 13: Factorial.cpp

```

1  #include <unistd.h>
2  #include <iostream>
3  #include <vector>
4  #include <sys/wait.h>
5
6  int factorial(int number){
7      if (number < 0) {
8          std::cout << "the number must be larger or equal to 0 !" << std::endl;
9          return -1;
  
```

```

10     }
11     if (number == 0) return 1;
12     int result = 1;
13     for(int i = 1; i <= number; i++){
14         result *= i;
15     }
16     return result;
17 }
18
19 int main(){
20
21     std::vector<int> numbers = {3,4,2,0,5};
22
23     std::vector<int> parentReadings;
24     std::vector<pid_t> pids;
25
26     for(int n : numbers){
27         int processPipe[2]; // read end [0] -> write end [1]
28
29         if (pipe(processPipe) == -1){
30             perror("pipe");
31             return 1;
32         }
33
34         pid_t pid = fork();
35
36         if(pid == 0){
37             close(processPipe[0]);
38
39             int result = factorial(n);
40             write(processPipe[1], &result, sizeof(int));
41
42             close(processPipe[1]);
43             _exit(0);
44
45         }else{
46             close(processPipe[1]);
47             parentReadings.push_back(processPipe[0]);
48             pids.push_back(pid);
49         }
50     }
51
52     std::vector<int> results(numbers.size());
53
54     for(int i = 0; i < numbers.size(); i++){
55         read(parentReadings[i], &results[i], sizeof(int));
56         close(parentReadings[i]);
57         waitpid(pids[i], nullptr, 0);
58
59         std::cout << "Factorial of " << numbers[i] << " = " << results[i] << std::
endl;
60     }
61
62     return 0;
63 }
64

```

Listing 14: Output

```

1      Factorial of 3 = 6
2      Factorial of 4 = 24
3      Factorial of 2 = 2
4      Factorial of 0 = 1
5      Factorial of 5 = 120
6

```

Question a Here each child gets its own copy of the address space because we are using a multiprocess approach, so nothing is shared unless we explicitly say so. This is the biggest difference from the threaded version, where every thread sees the same memory automatically.

Question b All local variables are private to each process, including its own copy of the list we initialized, the loop variable and the result. Only the pipe file descriptors connect the parent and the children.

Question c Results are communicated back through a dedicated pipe per child. The child writes its result into the write end with `write()`, and the parent reads it from the read end with `read()`, then calls `waitpid()` so no child is left as a zombie.

Question d The overhead in this exercise is the cost of `fork()` setting up the address space, plus the pipe creation and the `read/write` system calls for every single child. This is much higher than just passing a value between threads through shared memory.

9 Exercise 7 - Matrix Multiplication Using Multiprocessing

9.1 Part A - Theoretical Complexity vs. Sequential Measurements

The classical triple-loop matrix multiplication has sequential work $W(n) = \Theta(n^3)$. I measured the sequential execution time $T_1(n)$ for $n \in \{100, 200, 400, 800\}$ and compared it to a normalized theoretical model $T_{model}(n) = C \cdot n^3$, where $C = T_1(n_0)/n_0^3$ with $n_0 = 100$.

Listing 15: matrix_mult.cpp (sequential + theoretical model part)

```

1      #include <iostream>
2      #include <vector>
3      #include <random>
4      #include <chrono>
5
6      void mult(int matrixSize, int startRow, int endRow,
7      const std::vector<std::vector<int>>> &A,
8      const std::vector<std::vector<int>>> &B,
9      std::vector<std::vector<int>>> &C){
10
11      for(int i = startRow; i < endRow; i++){
12          for(int j = 0; j < matrixSize; j++){
13              int sum = 0;
14              for(int k = 0; k < matrixSize; k++){
15                  sum += A[i][k] * B[k][j];
16              }
17              C[i][j] = sum;
18          }
19      }
20  }
21

```

```

22 int main(){
23     std::random_device rd;
24     std::mt19937 gen(rd());
25     std::uniform_int_distribution<int> distrib(-10, 10);
26
27     std::vector<int> matrixSizes = {100, 200, 400, 800};
28     std::vector<double> T1;
29
30     for (int n : matrixSizes) {
31         std::vector<std::vector<int>> seqA(n, std::vector<int>(n));
32         std::vector<std::vector<int>> seqB(n, std::vector<int>(n));
33         std::vector<std::vector<int>> seqC(n, std::vector<int>(n, 0));
34
35         for (int i = 0; i < n; i++)
36             for (int j = 0; j < n; j++) {
37                 seqA[i][j] = distrib(gen);
38                 seqB[i][j] = distrib(gen);
39             }
40
41         auto start = std::chrono::steady_clock::now();
42         mult(n, 0, n, seqA, seqB, seqC);
43         auto end = std::chrono::steady_clock::now();
44
45         std::chrono::duration<double> timeElapsed = end - start;
46         T1.push_back(timeElapsed.count());
47
48         printf("n=%d -> T1=%fs\n", n, timeElapsed.count());
49     }
50
51     double n0 = matrixSizes[0];
52     double C_const = T1[0] / (n0 * n0 * n0);
53
54     for (size_t i = 0; i < matrixSizes.size(); i++) {
55         double Tmodel = C_const * matrixSizes[i] * matrixSizes[i] * matrixSizes[i];
56         printf("n=%d -> T1=%f, Tmodel=%f\n", matrixSizes[i], T1[i], Tmodel);
57     }
58
59     for (size_t i = 0; i + 1 < matrixSizes.size(); i++) {
60         if (matrixSizes[i+1] == 2 * matrixSizes[i]) {
61             double R = T1[i+1] / T1[i];
62             printf("R(%d->%d) = %f (ideal: 8.0)\n", matrixSizes[i], matrixSizes[i+1],
63                 R);
64         }
65     }
66     return 0;
67 }
68

```

n	$T_1(n)$ (s)	$T_{model}(n) = C n^3$ (s)
100	0.000314	0.000314
200	0.002983	0.002512
400	0.042819	0.020100
800	0.383426	0.160797

Table 4: Measured sequential time vs. normalized $\Theta(n^3)$ model

Doubling step	$R(n) = T_1(2n)/T_1(n)$ (ideal: 8.0)
100 → 200	9.499502
200 → 400	14.352585
400 → 800	8.954511

Table 5: Doubling test results

Looking at Table 5, the measured ratios do not sit exactly on the ideal 8.0, and the deviation is not random either. At the smaller sizes the fixed costs like allocating the matrices and generating the random numbers still make up a noticeable part of the total time, so the pure n^3 growth is not the only thing we are measuring. The biggest outlier is $R(200 \rightarrow 400) = 14.35$, which is most likely a cache threshold being crossed somewhere around $n = 400$ where the matrices stop fitting comfortably in cache. The closest one to the theory is $R(400 \rightarrow 800) = 8.95$, and that makes sense, because at larger sizes the real n^3 work dominates over all these fixed and cache related effects.

9.2 Part B - Parallel Scaling

The matrix is fixed at $n = 800$ and split into row-blocks, one block per worker. This was implemented twice: once with `std::thread` and once with real `fork()`-based multiprocessing and a shared-memory result matrix (via `mmap`), since the exercise specifically asks for *processes*.

Listing 16: `matmul_mp.cpp` (process-based version, following the professor's hint)

```

1  #include <iostream>
2  #include <vector>
3  #include <random>
4  #include <chrono>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <sys/mman.h>
8
9  void mult(int matrixSize, int startRow, int endRow,
10 const std::vector<std::vector<int>>& &A,
11 const std::vector<std::vector<int>>& &B,
12 int* result) {
13
14     for (int i = startRow; i < endRow; i++) {
15         for (int j = 0; j < matrixSize; j++) {
16             int sum = 0;
17             for (int k = 0; k < matrixSize; k++) {
18                 sum += A[i][k] * B[k][j];
19             }
20             result[i * matrixSize + j] = sum;
21         }
22     }
23 }
24
25 int main() {
26     std::random_device rd;
27     std::mt19937 gen(rd());
28     std::uniform_int_distribution<int> distrib(-10, 10);
29
30     int matrixSize = 800;
31     std::vector<int> numberOfProcesses = {1, 2, 4, 5, 10};
32
33     std::vector<std::vector<int>>& A(matrixSize, std::vector<int>(matrixSize));

```

```

34     std::vector<std::vector<int>> B(matrixSize, std::vector<int>(matrixSize));
35
36     for (int i = 0; i < matrixSize; i++)
37     for (int j = 0; j < matrixSize; j++) {
38         A[i][j] = distrib(gen);
39         B[i][j] = distrib(gen);
40     }
41
42     for (int n : numberOfProcesses) {
43
44         int* result = static_cast<int*>(mmap(nullptr, sizeof(int) * matrixSize *
matrixSize,
45         PROT_READ | PROT_WRITE,
46         MAP_SHARED | MAP_ANONYMOUS, -1, 0));
47
48         int rowsPerProcess = matrixSize / n;
49         int remainder = matrixSize % n;
50
51         auto start = std::chrono::steady_clock::now();
52
53         std::vector<pid_t> children;
54         int currentRow = 0;
55         for (int i = 0; i < n; i++) {
56             int rowsForThisProcess = 0;
57             if (remainder != 0) {
58                 rowsForThisProcess = rowsPerProcess + 1;
59                 remainder--;
60             } else {
61                 rowsForThisProcess = rowsPerProcess;
62             }
63
64             pid_t pid = fork();
65
66             if (pid == 0) {
67                 mult(matrixSize, currentRow, currentRow + rowsForThisProcess, A, B,
result);
68                 _exit(0);
69             } else if (pid > 0) {
70                 children.push_back(pid);
71             } else {
72                 std::cerr << "Fork Failed" << std::endl;
73                 return 1;
74             }
75
76             currentRow += rowsForThisProcess;
77         }
78
79         for (const auto& child : children) waitpid(child, nullptr, 0);
80
81         auto end = std::chrono::steady_clock::now();
82         std::chrono::duration<double> timeElapsed = end - start;
83         printf("Number of processes = %d\nTime elapsed in seconds = %fs\n", n,
timeElapsed.count());
84
85         munmap(result, sizeof(int) * matrixSize * matrixSize);
86     }
87
88     return 0;

```

p	T_p threads (s)	S_p threads	T_p processes (s)	S_p processes	E_p processes
1	0.387666	1.00	0.412830	1.00	1.00
2	0.209656	1.85	0.269791	1.53	0.77
4	0.125025	3.10	0.133150	3.10	0.78
5	0.112695	3.44	0.119772	3.45	0.69
10	0.066151	5.86	0.069175	5.97	0.60

Table 6: Matrix multiplication ($n = 800$): threads vs. processes

I also separately confirmed that for a small matrix ($n = 100$), adding more workers made execution *slower*, not faster, because the creation overhead exceeded the tiny amount of real computation involved. So parallelising only really pays off once the problem itself is big enough. Now lets answer the questions asked in the lab document.

Question a The measured T_p values along with the calculated S_p and E_p for both the thread and the process versions are given in Table 6. At $p = 10$ we reached a speedup of about 5.9 with an efficiency of around 0.60.

Question b In this experiment yes, the time kept going down all the way up to $p = 10$. But the gains get smaller and smaller each time, which you can see from the efficiency column dropping steadily from 1.00 to 0.60 as we add more workers.

Question c My machine has 16 logical processors (Intel Core Ultra 9 285H), so at $p = 10$ we are still below that limit and the drop in efficiency is not caused by running out of cores.

Question d The biggest one here is memory bandwidth and cache contention, since matrix multiplication is memory-bound and not purely CPU-bound, so all workers end up fighting over the same memory. On top of that the process version also pays for `fork()` copying the address space and for setting up the `mmap` shared region, which is why it came out slightly slower than the thread version at every p .

Question e Because the cost of creating a worker is more or less fixed, but the amount of useful work each worker gets shrinks as the matrix gets smaller. I tested this separately with $n = 100$ and adding threads actually made it *slower*, since the creation overhead was bigger than the tiny computation itself.

Question f It matches what we already saw in Part A: the measured growth follows $\Theta(n^3)$ reasonably well at larger sizes, but deviates at small and medium n where fixed overhead and cache effects still dominate the total time.

Question g Memory bandwidth is a shared and limited resource, so once several workers are all hammering the same memory at once they start slowing each other down. Adding more workers still helps, but each extra one contributes less than the previous, which is exactly why the measured curve bends away from the ideal $S_p = p$ line.

10 Exercise 8 - Barrier Synchronization

As a small, realistic example I modeled a simplified game/simulation frame update: in Phase 1 every object updates its own position, then all threads meet at a `std::barrier`, and only in Phase 2 - once every position is guaranteed up to date - do the threads check for collisions between objects.

Listing 17: Barrier.cpp

```

1  #include <iostream>
2  #include <thread>
3  #include <barrier>
4  #include <mutex>
5  #include <vector>
6  #include <chrono>
7
8  const int NUMBER_OF_OBJETCS = 4;
9  std::vector<double> positions(NUMBER_OF_OBJETCS);
10 std::vector<double> velocities = {1.,2.5,-1.5,.5};
11 std::mutex coutMutex;
12
13 void update(int objectID, std::barrier<>& sync){
14     // |----- PHASE 1 -----|
15     // | Update the position of objects |
16     // |-----|
17
18     positions[objectID] += velocities[objectID];
19     {
20         std::lock_guard<std::mutex> lock(coutMutex);
21         std::cout << "Object " << objectID << " updated position -> " << positions[
objectID] << std::endl;
22     }
23
24     std::this_thread::sleep_for(std::chrono::milliseconds(50 * (objectID + 1)));
25
26     sync.arrive_and_wait(); // BARRIER
27
28     // |----- PHASE 2 -----|
29     // | Now the new positions are updated, we can check for collisions |
30     // |-----|
31     for (int other = 0; other < NUMBER_OF_OBJETCS; other++) {
32         if (other == objectID) continue;
33
34         double distance = std::abs(positions[objectID] - positions[other]);
35
36         if (distance < 1.0) {
37             std::lock_guard<std::mutex> lock(coutMutex);
38             std::cout << " [Object " << objectID << "] possible collision with
Object "
39             << other << " (distance=" << distance << ")" << std::endl;
40         }
41     }
42 }
43
44 int main(){
45     for(int i = 0; i < NUMBER_OF_OBJETCS; i++){
46         positions[i] = i * 2.0;
47     }
48
49     std::barrier sync(NUMBER_OF_OBJETCS);
50
51     std::vector<std::thread> threads;
52     for(int i = 0; i < NUMBER_OF_OBJETCS; i++){
53         threads.emplace_back(update, i, std::ref(sync));
54     }
55     for(auto& t : threads){

```

```

56     t.join();
57 }
58
59     std::cout << "Frame update complete." << std::endl;
60     return 0;
61 }
62

```

Listing 18: Output

```

1     Object 1 updated position -> 4.5
2     Object 2 updated position -> 2.5
3     Object 3 updated position -> 6.5
4     Object 0 updated position -> 1
5     Frame update complete.
6

```

An artificial, per-object sleep ($50 * (\text{objectID} + 1)$ ms) was added purely so the effect of the barrier is easy to observe: objects finish Phase 1 at clearly different times, yet the "Frame update complete." line - and any Phase 2 output - only ever appears after every single object has finished updating its position. In this particular run no collision was reported, simply because none of the four objects ended up closer than 1.0 to each other. A global `std::mutex` around `std::cout` was also required, otherwise the interleaved prints from different threads produced garbled, unreadable output.

Question A mutex and a barrier solve completely different problems even though both of them are called synchronization tools. A mutex is about *exclusion*: it makes sure only one thread at a time is inside a critical section, and the other threads just queue up behind it. A barrier is about *meeting up*: nobody is excluded from anything, instead every thread has to reach the same point before any of them is allowed to continue, and then they all get released together.

Due to my heavy interest in Computer Graphics and Game Development I thought it would be right to give a realistic example from that area, which is exactly the one I used above: a frame update in a game or physics engine. Each object's new position is calculated in parallel, but collision detection cannot start until *every* object has finished updating, because a thread doing collision checks needs to read the other objects' positions too. Without a barrier a fast thread could easily reach the collision phase while a slower one has not written its new position yet, so it would compare against a stale value and possibly report a collision that does not exist, or miss one that does. The barrier guarantees that Phase 2 only ever sees fully updated data.

11 Exercise 9 - Parallel Map-Reduce

A list of 1000 random integers is processed with a map phase (squaring) and a reduce phase (summation), first sequentially and then using `fork()`-based multiprocessing with a shared-memory array of partial sums.

Listing 19: map_reduce.cpp

```

1     #include <iostream>
2     #include <vector>
3     #include <unistd.h>
4     #include <sys/wait.h>
5     #include <sys/mman.h>
6     #include <chrono>

```

```

7      #include <random>
8
9      std::vector<int> map_phase(const std::vector<int>& numberList){
10         std::vector<int> result;
11         for(int n : numberList){
12             result.push_back(n*n);
13         }
14         return result;
15     }
16
17     int reduce_phase(const std::vector<int>& squared){
18         int sum = 0;
19         for(int n : squared){
20             sum += n;
21         }
22         return sum;
23     }
24
25     int main(){
26         std::random_device rd;
27         std::mt19937 gen(rd());
28         std::uniform_int_distribution<int> distrib(0, 1000);
29
30         std::vector<int> numberList;
31         for(int i = 0; i < 1000; i++){
32             numberList.push_back(distrib(gen));
33         }
34
35         //-----SEQUENTIAL-----
36         auto timerStart = std::chrono::steady_clock::now();
37         std::vector<int> squaredNumberList = map_phase(numberList);
38         int sum = reduce_phase(squaredNumberList);
39         auto timerEnd = std::chrono::steady_clock::now();
40         std::chrono::duration<double> timeElapsed = timerEnd - timerStart;
41
42         std::cout << "SEQUENTIAL : Sum of squares = " << sum << std::endl;
43         std::cout << "                Time Elapsed   = " << timeElapsed.count() << "s"
44         << std::endl;
45
46         //-----PARALLEL-----
47         int numberOfProcesses = 4;
48
49         long long* partialSums = static_cast<long long*>(mmap(nullptr, sizeof(long
50 long) * numberOfProcesses,
51 PROT_READ | PROT_WRITE,
52 MAP_SHARED | MAP_ANONYMOUS, -1, 0));
53         int itemsPerProcess = numberList.size() / numberOfProcesses;
54         int remainder = numberList.size() % numberOfProcesses;
55
56         std::vector<pid_t> children;
57         int currentStart = 0;
58
59         auto parallelStart = std::chrono::steady_clock::now();
60
61         for(int i = 0; i < numberOfProcesses; i++){
62             int itemsForThisProcess = itemsPerProcess;
63             if(remainder != 0){
64                 itemsForThisProcess++;

```

```

63     remainder--;
64 }
65 int start = currentStart;
66 int end = currentStart + itemsForThisProcess;
67 currentStart = end;
68
69 pid_t pid = fork();
70
71 if(pid == 0){
72     std::vector<int> subList(numberList.begin() + start, numberList.begin() +
end);
73     std::vector<int> squaredSubList = map_phase(subList);
74     long long partialSum = reduce_phase(squaredSubList);
75
76     partialSums[i] = partialSum;
77     _exit(0);
78 }else{
79     children.push_back(pid);
80 }
81 }
82 for (const auto& child : children) waitpid(child, nullptr, 0);
83
84 long long totalSum = 0;
85 for (int i = 0; i < numberOfProcesses; i++) totalSum += partialSums[i];
86
87 auto parallelEnd = std::chrono::steady_clock::now();
88 std::chrono::duration<double> parallelTimeElapsed = parallelEnd -
parallelStart;
89
90 std::cout << "PARALLEL    : Sum of squares = " << totalSum << std::endl;
91 std::cout << "                Time Elapsed   = " << parallelTimeElapsed.count()
<< "s" << std::endl;
92
93 munmap(partialSums, sizeof(long long) * numberOfProcesses);
94
95 return 0;
96 }
97

```

Listing 20: Output

```

1    SEQUENTIAL : Sum of squares = 334449794
2    Time Elapsed   = 0.0000022s
3    PARALLEL    : Sum of squares = 334449794
4    Time Elapsed   = 0.000333254s
5

```

Both versions computed the exact same sum, confirming correctness. However, the parallel version was about 150 times *slower* than the sequential one at this input size.

Question From my measurements I was able to see exactly what the lab exercise wanted me to realize. The sequential growth here is $\Theta(n)$, since we do n independent squarings and then combine n values. The parallel version came out slower because the real computation (squaring and summing 1000 integers) is tiny compared to the cost of `fork()`, `mmap` and `waitpid` for 4 processes, so the process creation overhead never gets amortized. To find the crossover point where multiprocessing actually starts paying off, the same test would need to be re-run with a much larger n , something like 10 million elements.

12 Exercise 10 - Using a Thread Pool for I/O-Bound Tasks

A small downloader was built with libcurl, fetching 12 files from a local Python HTTP server (python3 -m http.server 8001) so results are reproducible and no public site is overloaded. A sequential version and a thread-pool version (pool sizes 1, 2, 4, 5, 6, 8, 12) were compared.

Listing 21: thread_pool_IO.cpp

```
1  #include <curl/curl.h>
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <chrono>
6  #include <thread>
7
8  size_t writeCall(void* contents, size_t size, size_t nmemb, std::string* output)
9  {
10     size_t totalSize = size * nmemb;
11     output->append((char*)contents, totalSize);
12     return totalSize;
13 }
14
15 void downloadURL(const char* url, std::string& result) {
16     CURL* curl = curl_easy_init(); // each thread needs its OWN handle - CURL* is
17     not thread-safe to share
18     if (curl) {
19         curl_easy_setopt(curl, CURLOPT_URL, url);
20         curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, writeCall);
21         curl_easy_setopt(curl, CURLOPT_WRITEDATA, &result);
22
23         CURLcode res = curl_easy_perform(curl);
24         if (res != CURLE_OK) {
25             std::cerr << "curl error: " << curl_easy_strerror(res) << std::endl;
26         }
27
28         curl_easy_cleanup(curl);
29     }
30 }
31
32 int main(){
33     curl_global_init(CURL_GLOBAL_DEFAULT);
34
35     std::vector<std::string> URLs;
36     for (int i = 1; i <= 12; i++) {
37         URLs.push_back("http://127.0.0.1:8001/file" + std::to_string(i) + ".txt");
38     }
39
40     std::vector<int> poolSizes = {1, 2, 4, 5, 6, 8, 12};
41
42     for (int poolSize : poolSizes) {
43         std::vector<std::thread> threads;
44         std::vector<std::string> results(URLs.size());
45
46         int urlsPerThread = URLs.size() / poolSize;
47         int remainder = URLs.size() % poolSize;
48
49         auto start = std::chrono::steady_clock::now();
50
51         int currentIndex = 0;
```

```

50     for (int i = 0; i < poolSize; i++) {
51         int urlsForThisThread = 0;
52         if (remainder != 0) {
53             urlsForThisThread = urlsPerThread + 1;
54             remainder--;
55         } else {
56             urlsForThisThread = urlsPerThread;
57         }
58
59         int startIdx = currentIndex;
60         int endIdx = currentIndex + urlsForThisThread;
61         currentIndex = endIdx;
62
63         threads.emplace_back([startIdx, endIdx, &URLs, &results]() {
64             for (int j = startIdx; j < endIdx; j++) {
65                 downloadURL(URLs[j].c_str(), results[j]);
66             }
67         });
68     }
69
70     for (auto& t : threads) t.join();
71
72     auto end = std::chrono::steady_clock::now();
73     std::chrono::duration<double> timeElapsed = end - start;
74
75     std::cout << "Pool size = " << poolSize
76     << " -> Time elapsed = " << timeElapsed.count() << "s" << std::endl;
77 }
78
79 curl_global_cleanup();
80 return 0;
81 }
82

```

Pool size	Time elapsed (s)
1	0.00883
2	0.00732
4	0.00559
5	0.00531
6	0.00972
8	0.00582
12	1.02392

Table 7: Thread pool download times (12 local files)

I came across a really interesting finding while implementing this exercise. Normally you would expect the time to keep going down as the pool size increases, but that was not the case at all once we hit pool size 12, and this result was not noise either since it was reproducible across multiple runs. The cause was traced to Python's default TCP connection backlog, confirmed with:

Listing 22: Checking the server's backlog size

```

1     python3 -c "import socketserver; print(socketserver.TCPServer.
      request_queue_size)"
2

```

which returns 5. When 12 threads open connections almost simultaneously, more connections arrive than the server's backlog of 5 can hold, some SYN packets get dropped, and the OS's TCP retransmission timeout (approximately 1 second) explains the observed slowdown. Using 127.0.0.1 instead of `localhost` ruled out DNS resolution as a possible cause, since the behaviour did not change at all.

Question A thread pool works well here because the threads are not really doing heavy work, they are just waiting for the network most of the time. So while one thread waits, the others can send their own requests, which is why we can use more threads than we have cores and still benefit.

But it only helps up to a point. As Table 7 shows, pool sizes 1 to 8 all gave similar times, but at pool size 12 it jumped to about 1 second, which is around 140 times slower. The reason was the server, not my code: Python's `http.server` only queues 5 connections by default, so when 12 threads connect at once some of them get dropped and have to retry about a second later. That is our saturation point.

13 Exercise 11 - Using a Process Pool for the Same I/O-Bound Task

The same downloader was reimplemented using `fork()` instead of `std::thread`, with each child process handling its own share of the URL list.

Listing 23: `process_pool_IO.cpp`

```
1  #include <curl/curl.h>
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <chrono>
6  #include <unistd.h>
7  #include <sys/wait.h>
8
9  size_t writeCall(void* contents, size_t size, size_t nmemb, std::string* output)
10 {
11     size_t totalSize = size * nmemb;
12     output->append((char*)contents, totalSize);
13     return totalSize;
14 }
15
16 void downloadURL(const char* url) {
17     CURL* curl = curl_easy_init();
18     std::string result;
19
20     if (curl) {
21         curl_easy_setopt(curl, CURLOPT_URL, url);
22         curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, writeCall);
23         curl_easy_setopt(curl, CURLOPT_WRITEDATA, &result);
24
25         CURLcode res = curl_easy_perform(curl);
26         if (res != CURLE_OK) {
27             std::cerr << "curl error: " << curl_easy_strerror(res) << std::endl;
28         }
29
30         curl_easy_cleanup(curl);
31     }
```

```

32
33 int main(){
34     curl_global_init(CURL_GLOBAL_DEFAULT);
35
36     std::vector<std::string> URLs;
37     for (int i = 1; i <= 12; i++) {
38         URLs.push_back("http://127.0.0.1:8001/file" + std::to_string(i) + ".txt");
39     }
40
41     std::vector<int> poolSizes = {1, 2, 4, 6, 12};
42
43     for(int poolSize : poolSizes){
44         int urlsPerProcess = URLs.size() / poolSize;
45         int remainder = URLs.size() % poolSize;
46
47         std::vector<pid_t> children;
48         int currentIndex = 0;
49
50         auto start = std::chrono::steady_clock::now();
51
52         for (int i = 0; i < poolSize; i++) {
53             int urlsForThisProcess = 0;
54             if (remainder != 0) {
55                 urlsForThisProcess = urlsPerProcess + 1;
56                 remainder--;
57             } else {
58                 urlsForThisProcess = urlsPerProcess;
59             }
60
61             int startIdx = currentIndex;
62             int endIdx = currentIndex + urlsForThisProcess;
63             currentIndex = endIdx;
64
65             pid_t pid = fork();
66
67             if (pid == 0) {
68                 curl_global_init(CURL_GLOBAL_DEFAULT); // each process needs its OWN
libcurl init
69                 for (int j = startIdx; j < endIdx; j++) {
70                     downloadURL(URLs[j].c_str());
71                 }
72                 curl_global_cleanup();
73                 _exit(0);
74             } else {
75                 children.push_back(pid);
76             }
77         }
78
79         for (const auto& child : children) waitpid(child, nullptr, 0);
80
81         auto end = std::chrono::steady_clock::now();
82         std::chrono::duration<double> timeElapsed = end - start;
83
84         std::cout << "Pool size = " << poolSize
85         << " -> Time elapsed = " << timeElapsed.count() << "s" << std::endl;
86     }
87
88     curl_global_cleanup();

```

```

89     return 0;
90 }
91

```

Method	Pool size	Time elapsed (s)
Sequential	1 (all 12 files)	0.00908
Process pool	12	0.00797
Thread pool	12	1.02392

Table 8: Sequential vs. thread pool vs. process pool at the saturation point

Unlike the thread pool, the process pool did *not* show a collapse at pool size 12: its time stayed in line with the other pool sizes and with the sequential baseline.

Question a What was the fastest implementation here? In this exercise the process pool and the sequential version were both close to each other and clearly faster than the thread pool at pool size 12, for this local and very low latency setup.

Question b As expected the process pool used the most system resources, since each process gets its own full address space thanks to `fork()`, which is much heavier than a thread, and unlike threads they do not share memory by default.

Question c A multiprocess approach is often less attractive for I/O-bound work because the creation and context-switch costs are higher, and since there is no shared memory by default you also need some kind of IPC, which makes it harder to implement and maintain.

Question d Even though they are harder to implement there are still strong reasons to use processes. A process pool is preferable when isolation matters, for example when one worker crashing should not take down all the others. In this specific experiment it also happened to avoid the server backlog saturation that hit the thread pool, most likely because `fork()`'s slower startup naturally spread out the connection attempts over time instead of firing them all at once.

14 Conclusion

Going through all 11 exercises, the thing that stuck with me the most is that concurrency, parallelism and synchronization are three different things, even though people tend to use them as if they mean the same. Concurrency is about several things being *in progress* at the same time, which is what we saw in Exercise 1 where the two threads' outputs kept interleaving differently on every run. Parallelism is about several things actually *executing* at the same instant on different cores, which only really showed up once we had enough independent work to hand out, like the matrix multiplication in Exercise 7. And synchronization is the price you pay to keep all of that correct, which we felt directly in Exercise 4 where the mutex version was around 71 times slower but was the only one that ever gave the right answer.

The other big lesson for me was that more workers does not automatically mean faster. In Exercise 2 Part A adding a thread changed nothing because there was only one unit of work. In Exercise 9 the parallel map-reduce was about 150 times slower than the sequential version because squaring 1000 numbers is simply not worth four `fork()` calls. In Exercise 7 the small 100×100 matrix also ran faster sequentially. And in Exercise 10 the thread pool got dramatically worse at pool size 12, not because of anything in my code but because the server on the other side could only queue 5 connections. That last one was probably my favourite finding of the whole lab, because it showed me that you can be limited by something completely outside your own program, and you only find it if you actually go and check instead of guessing.

GitHub Repository

The full source code for all exercises (both C++ and Java implementations where applicable) is available at: <https://github.com/BoraYalcinn/Multithreading-Multicore-Architecture>